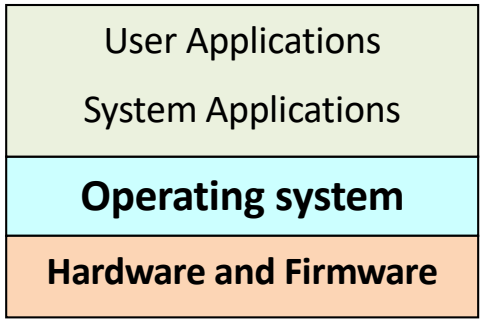
Systems Administration (ASIST)

Practical Theoretical Class 01

Server Operating Systems

Operating system

Operating System (**OS**) is a set of software components that enable you to establish a stable and homogeneous platform upon which system and user applications can operate.



The hardware that the operating system works on can be very diverse. The operating system is a layer of abstraction, thanks to it, applications do not need to be developed for specific types of hardware. **Applications indirectly use hardware through operating system.**

Some of the most important functions of an operating system are **process** and **thread** management, **central memory** (**RAM**) management, and **external device** management.

Operating system components that perform these three basic functions are what are commonly referred to as the **operating system kernel** or **kernel.**

Server Operating Systems

Although any operating system can be used to implement a server, there are versions of operating systems specifically designed for this purpose. Typically they are called ***server***.

In most cases, **server** versions are based on traditional (**desktop**) versions, often the **kernel** is the same.

Today, three families of server operating systems stand out:

Microsoft Windows Server – server version of Microsoft operating systems, the first was Windows NT Server.

Unix-based *open source* versions – stand out among others the family **LINUX** and **BSD** (NetBSD, FreeBSD and OpenBSD).

OS X – originally called MAC OS Server, there is currently only one version that is both server and desktop.

File Systems

Among the various types of external devices that an operating system manages are storage devices (***storage***). Unlike typical central memory (**RAM**) they are usually nonvolatile, ie stored data persists when the system shuts down.

A basic feature of a storage device is its capacity, usually expressed in bytes, each byte is stored in an addressable position of the device. Some devices may be read-only, such as non-rewritable CDs.

The operating system accesses individual bytes on the storage device, but this type of access is not suitable for applications.

To allow more expedient use by applications, the operating system establishes a **file system** over the storage device. This operation is called **formatting** and consists of organizing the storage device into folders (directories) and files.

Partitions and file systems

File system management is also a core function of any operating system and is integrated into the *kernel*.

Once the file system is created on the external storage device the operating system can provide file usage functions (open / create, read, write, close,…) that are much better suited for applications and easier to manage (permissions, concurrency ,…) by the kernel.

Many external storage devices are fixed (non-removable) disks: **HDD** (*Hard Disk Drive*) or **SSD** (*Solid State Drive*). Normally fixed disks **must be partitioned before they can be formatted**, even if only one partition is desired on the disk.

A partition is a continuous zone that starts and ends at well-defined disk positions and is independent of other partitions. For all intents and purposes it can be seen as a standalone disc.

A file system created in one partition is completely independent of file systems in other partitions.

Partitions table

The partition table defines the start and end positions of each partition, it is stored at the beginning of the disk itself in a zone called *Master Boot Record* (**MBR**).

The first thing an operating system does when it has access to a disk is to read the partition table. Also, the first step in setting up a new blank disk is to create a new partition table.

Once a partition is defined, it can be formatted using a particular file system type, however, once formatted, changing the partition (start and end positions) is no longer a trivial process.

Changing a formatted partition without further care will result in the complete loss of the file system there (data).

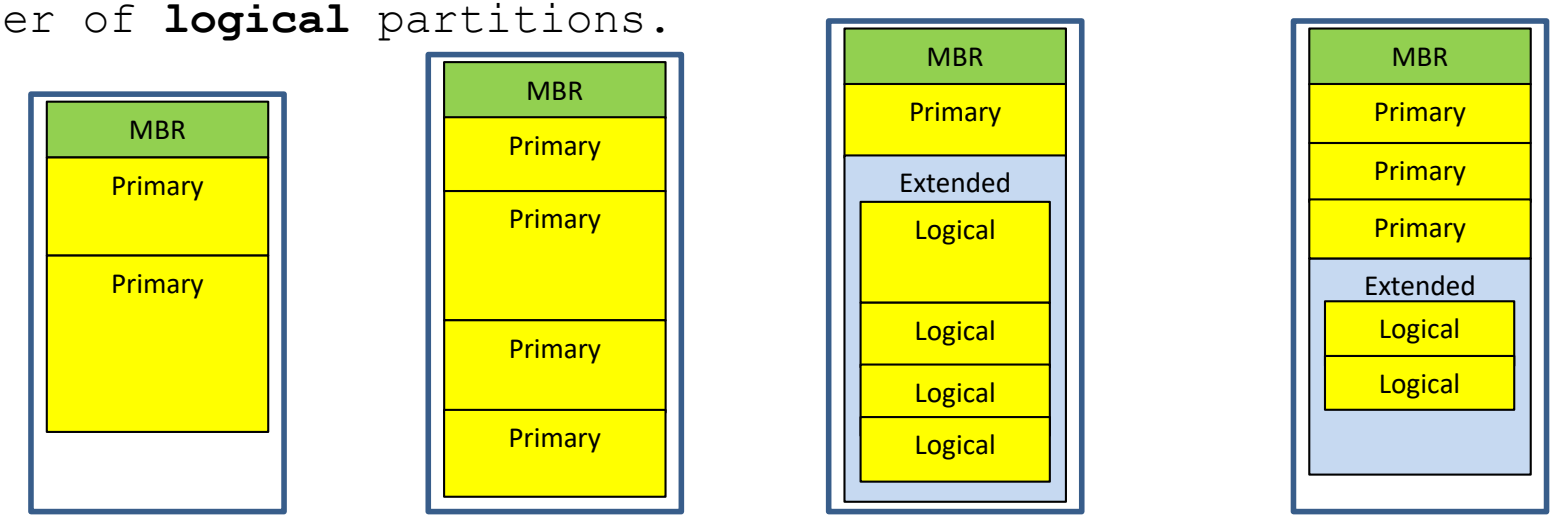
Changing a partition containing data (file system) involves moving the entire file system. It is usually simple to **expand a partition** (advance to the end position) as long as the next disk space is not allocated to any other partition.

MBR - Primary, Extended, and Logical Partitions

The MBR partition table has been inherited from the early days of personal computers and has many limitations. There are currently some alternatives, but trying to maintain compatibility with the MBR format.

The MBR table is static and has four entries that may or may not be defined. Therefore allows a maximum of 4 partitions. Of these, at most one can be ***extended***, the rest are ***primary***.

Extended partitions exist for the purpose of circumventing the limit of 4 partitions, an **extended** partition can contain an unlimited number of **logical** partitions.



Logical partitions can be used in the same way as **primary** partitions, for example it can be the operating system boot partition - assuming there is a *primary* partition containing the *boot loader*.

Disk and partition identification

Different operating systems use different ways of identifying disks and partitions, in this course we will focus our study on Unix (Linux) and MS-Windows systems.

On Linux systems the current disks (SCSI and SATA interfaces) are identified by **/dev/sda, /dev/sdb,**… according to the order in which they are detected by the *kernel*. Existing partitions on each disk are identified by sequential numeric suffixes beginning with one. For example /**dev/sdb5** identifies the fifth partition of the second disk.

On Windows systems disks are identified by numeric values starting at zero: **"Disk 0", "Disk 1" ,** …

As with Unix, partitions on each disk are identified by numeric values with the first partition being assigned the value 1. For example, the designation **"Disk 1 partition 3"** refers to the third partition on the second disk. On Windows systems the volume designation is used to refer to a partition. In order for a partition (volume) to be usable by applications it must also be **assigned a unique drive letter.**

Partition Table Management - Linux

On Linux the traditional command to manage the partition table is ***fdisk***. With this command you can interactively create and change partitions:

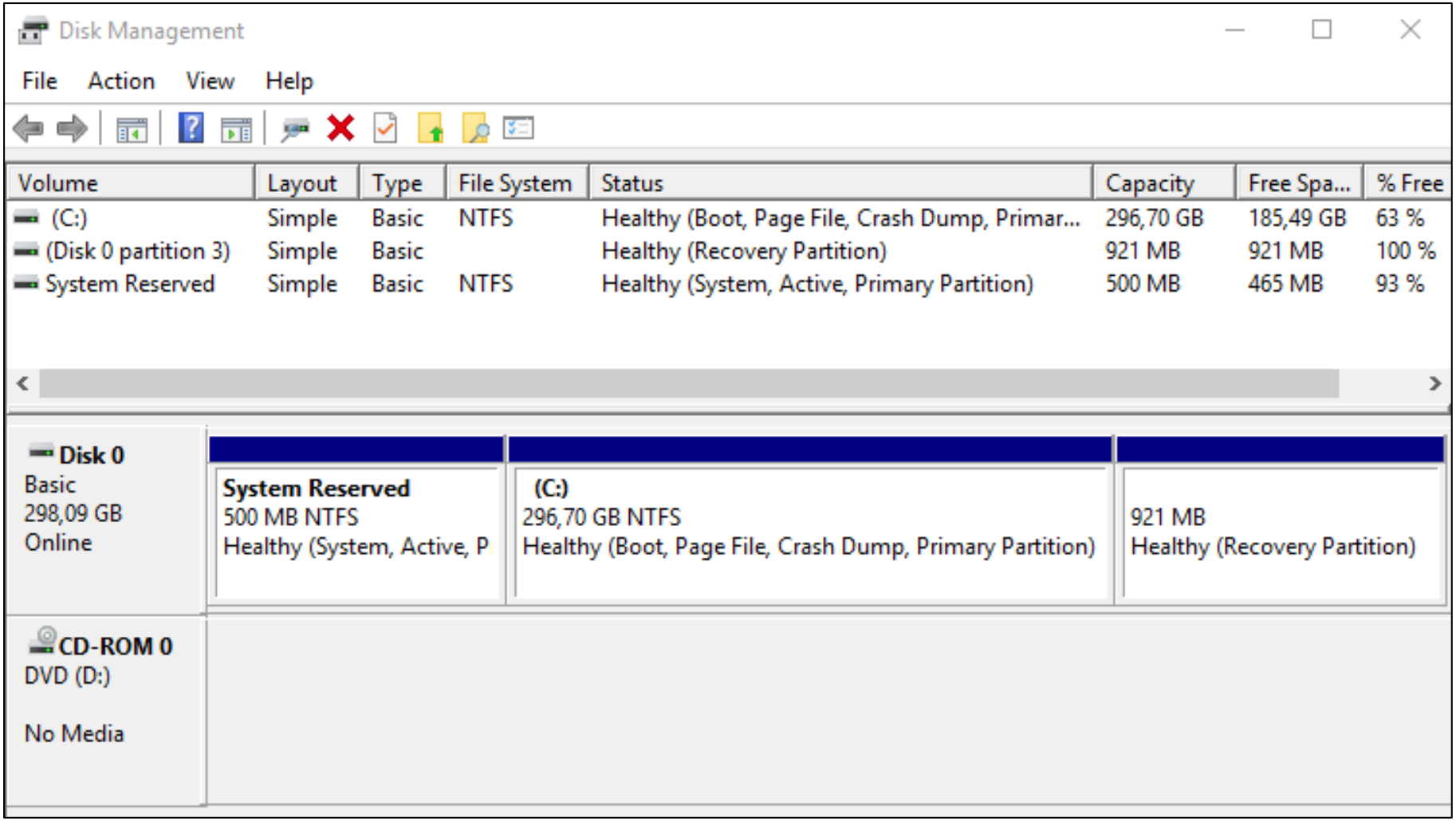
```
HOST1# fdisk /dev/sda
Command (m for help): p
Disk /dev/sda: 73.4 GB, 73407868928 bytes
255 heads, 63 sectors/track, 8924 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Device    Boot      Start     End       Blocks    Id        System
/dev/sda1 *         1         101       811251    83        Linux
/dev/sda2           102       8924      70870747+ 5         Extended
/dev/sda5           102       202       811251    82        Linux swap
/dev/sda6           203       711       4088511   83        Linux
/dev/sda7           712       4757      32499463+ 83        Linux
/dev/sda8           4758      8797      32451268+ 83        Linux
/dev/sda9           8798      8924      1020096   83        Linux
```

The above example shows (***fdisk* -p** argument) the disk partition table **/dev/sda**. As you can see a partition table contains a lot of information, in addition to the positions of each partition, also the file system type of each partition and the indication of what is the boot partition *(boot)*, in this case the primary partition **/dev/sda1.**

We can also see an *extended* partition (**/dev/sda2**) which contains the logical partitions **/dev/sda5** through **/dev/sda9.**

Partition Table Management - Windows

In Windows the **Disk Management** application allows to perform the same type of operations and additionally assign to a formatted volume a drive letter:



GUID Partition Table (GPT)

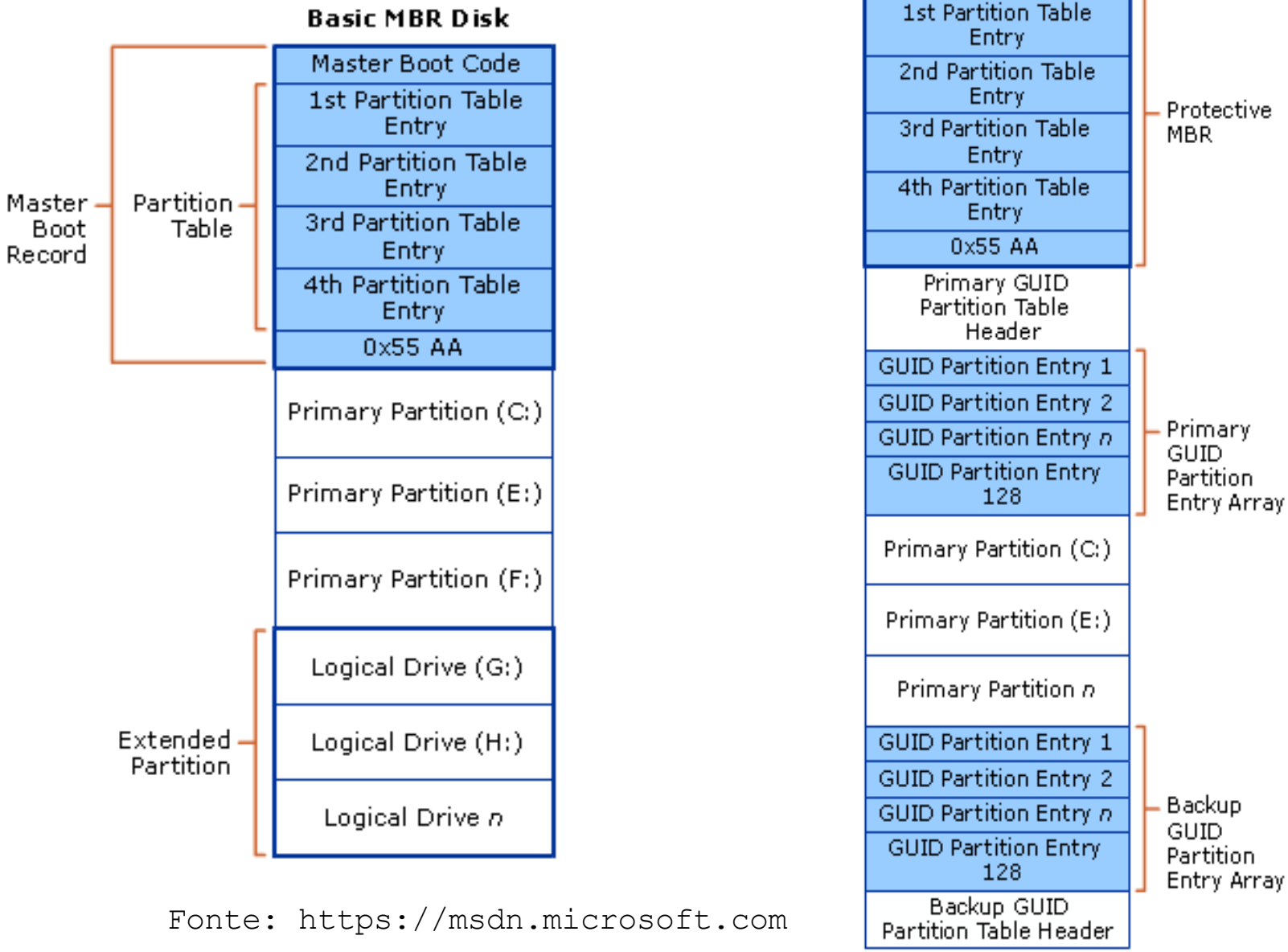
Although the MBR limit of four partitions per disk is exceeded by logical partitions, there are other problems.

With MBR, due to the shape of the addresses used to refer to disk positions, it is not possible to define partitions beyond the 2 terabyte position. If the disc is larger in size, it will be unused. Another problem is that there is only one copy of MBR, if it gets corrupted the reference to existing partitions is lost.

The GUID (*globally unique identifiers*) *Partition Table* (GPT) solves these problems. Supports up to 128 primary partitions, allows table integrity validation, and backs up to an alternate disk location.

GPT disks maintain MBR for limited compatibility (*protective MBR*) with non-GPT-aware operating systems.

MBR Partition Table and GPT



Fonte: https://msdn.microsoft.com

Partition Formatting

Defining a new partition or volume (for example using the fdisk command on Linux or the Disk Management application on Windows) is a simple partition table edit operation, it has no effect on the disk zone corresponding to that partition. In order to be used for data storage the disk zone referenced in the partition table must be prepared (**formatted**).

There are many file system formats, currently Microsoft Windows environments use **NTFS** (*New Technology File System*), on Linux the most used is **EXT4**.

The file system stores various information about objects, including their location within the partition and attributes such as name and **ACL** (permissions). All this information is stored in structures stored in the partition itself, the process of creating these structures is known as **format operation.**

After formatting operation a completely empty file system is obtained.

Windows – *Basic* and *Dynamic disks*

In Windows Server, disks are initially of type ***basic***, but can be converted to ***dynamic***. *Basic* disks use MBR partition table and GTP.

Disks converted to dynamic also have the MBR / GTP partition table, but contain only one partition called *Logical Disk Manager* (LDM). Linux systems also support LDM.

The *dynamic* disks allow several volumes settings multipartition not supported for *basic* disks. To encourage the use of *dynamic* disks, Microsoft no longer supports multi-partition volume configurations on basic disks.

Currently a basic disk can only contain *simple volumes*, however, a *simple volume* of a *basic* disk can be expanded as long as there is free space immediately following the corresponding partition and using the NTFS file system. Volumes defined on a *basic* disk are called *basic volumes*, volumes defined on a *dynamic* disk are called *dynamic volumes*.

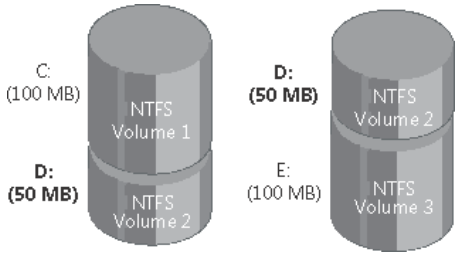
Dynamic volumes

Dynamic volume is located on a single disc but can be spread across several locations on the disc. If there is available space, unformatted or formatted as NTFS, it can always be expanded.

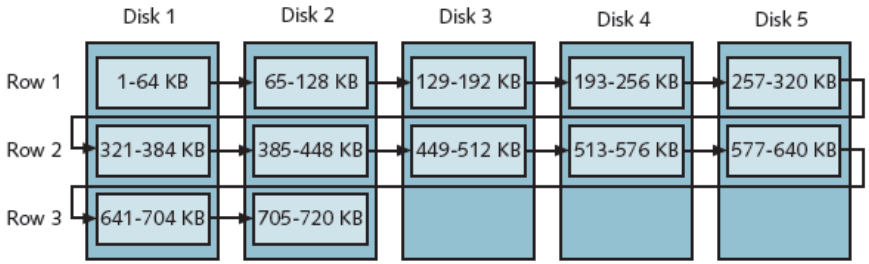
Spanned Volume is spread over more than one disc. Once space has been added to a spanned volume from a disc, it cannot be removed.

Striped Volume is spread over more than one disc, and in addition to obtaining a larger volume, it improves performance, corresponding to a **RAID 0** operating mode. Blocks of equal size are allocated to all the discs and the data is accessed transversally through the blocks on the different discs.

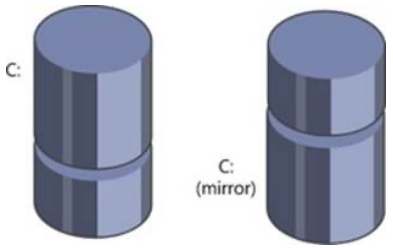
Mirrored volume is a simple volume that has an exact duplicate on another disc, corresponding to a configuration **RAID1**.



Spanned volume



Striped volume



Mirrored volume

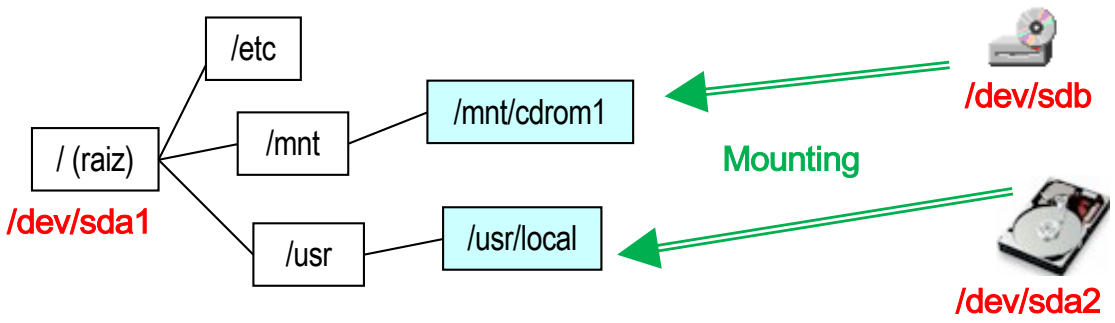
File system access

The operating system gives applications access to file systems resident on the various partitions of the various disks. For DOS/Windows systems drive letters are used for this purpose, the letters **A:** and **B:** have traditionally been reserved for floppy disks (disused) and so the available partitions (volumes) are identified by letters from **C:**.

Linux systems (and Unix in general) use a different philosophy, choosing a file system (partition) to take on the root role (***root***). Other existing file systems of whatever type are integrated into the root file system through a designated mount operation (***mount***).

The mount is a logical association created by the operating system between an empty directory and a file system residing on another partition or disk.

After mounting, the operating system redirects to the mounted partition file system all access to the empty directory.



Linux distributions

Although there is only one free, open source Linux *kernel* that is under continuous development, a large set of additional software is required to get a working server system. The complete set that allows you to install a Linux server is known as **distribution**. Because some of the required additional software may not be free, not all Linux distributions are free.

A key question that an administrator who wants to install a Linux server faces is "**What distribution to use?**". There are dozens of current distributions that can be grouped into families according to the initial distribution they were derived from. The **RedHat** family and the **Debian** family can be highlighted here.

Since the *kernel* is the same, the differences between families lie mainly in the application package manager used and the way system configuration files are organized. Certainly for an administrator who usually manages systems of a given family there will be a tendency to stay in the same family as he is already familiar with the commands and location of configuration files.

Linux distributions

Usually distributions are available in two versions: *server* and *desktop/workstation*.

In the case of Linux, the biggest difference between the two is that the server version does not include graphical interface support, this means that these features are available to the server role.

On a server we aim above all for stability and longevity, so we must opt for stable, update-supported distributions for a few years.

For example **Ubuntu** distributions (Debian family) come in **LTS** (Long Term Support) version, are available free of charge every 2 years and updates are provided for 5 years.

Also noteworthy are the non-free distributions that include direct technical support, such as *Red Hat Enterprise Linux* (RHEL).

The inherent architecture of a Linux server has the disadvantages of requiring additional packages for some group management tasks (such as authentication, for example) and possible incompatibility between packages. The advantages can point to no requirement for a dedicated server.

Windows Server operating systems

Server versions of Microsoft operating systems were introduced in 1993 with the first Windows NT Server. The latest version is Windows 2019 Server.

Windows Server represents an alternative to the workgroup philosophy (***workgroup***). Until Windows NT Server, the only network environment provided by Microsoft was ***peer-to-peer*** with only workstations that could share resources with each other (***workgroup***).

Server versions allow **server-based** operation where dedicated servers exist and the concept of **workgroup** is then converted to **domain**.

Server-based architecture has notable advantages:

- Centralized and single authentication (domain).
- Increased security.
- Centralization of resources and operations.

Disadvantages include the need for at least one dedicated server and increased administration complexity.

Windows Server operating systems - *editions*

Windows Server is not free, each version comes in multiple *editions*, each of which may include different roles/features, limits on the number of users, and limits on the maximum amount of memory (RAM) and number of CPUs that can be used. The price of each edition may differ substantially.

For example Windows 2019 Server has the following *editions*:

- Windows Server 2019 Essentials
- Windows Server 2019 Standard
- Windows Server 2019 Datacenter
- Windows Server 2019 Hyper-V

To be able to compete with non-graphical server operating systems such as Linux, the latest versions of Windows Server allow a ***Server Core*** installation. With a *Server Core* installation, no graphical interface is installed and all server management will need to be done through the command line on the console.

Another installation option available on 2019 Server is ***Nano Server***, which still consumes less resources than *Server Core*, but in this case all administration has to be done over the network as console access is not even possible.

Operating system installation

While it is possible to start the process of installing an operating system on a machine already in operation, the most common way is to boot the machine from a ready-made external storage device, typically a CD / DVD or a USB *flash drive*.

The boot device contains an elementary operating system (may not be the operating system to be installed), supporting the applications to install.

The installation process triggered in this way is almost fully automated, the applications that install it analyze the hardware and try to deduce the most appropriate settings.

You will usually be asked for some information and confirmation from the user, such as disk usage and partitioning.

Depending on the installation program, we may in some cases indicate our level of knowledge about the operating system and installation process and thus change the amount of information that will be requested and not decided automatically. For example the "*expert*" installation mode.

Operating system installation - partitions

In most cases the installer makes the right decisions, but it lacks information on the exact purposes intended for the server being installed.

It is true that most of the settings made in the installation process can easily be changed later when the server is operational. One thing to keep in mind is partitioning the disks, changing partitions that are in use is not a simple task, so we should first define the partitions according to long term goals.

Partitions are static boundaries, if a partition is too small, when it becomes full there is no simple solution to expand it if there is another partition to follow on disk.

On Unix / Linux systems the quota limits (maximum space a user can occupy) are set for each partition / file system, so if multiple quota values are required for different functions (web, mail, etc.) there must be a separate partition for each function.

On Windows systems, global quotas can be set for a folder (folder quotas), but do not refer to individual users. User quotas are set for each volume (disk quotas).

Operating system installation - hardware

Today most servers are installed on virtualization platforms as virtual servers and not on physical hardware.

An advantage of using a virtual machine is that it is designed specifically to support the intended operating system and as such all hardware devices being simulated are easily recognized by the installer.

When installing an operating system on real hardware everything can be complicated if the installation system does not have proper drivers for that hardware.

Notably, if the installation system does not have a suitable driver for the disk controller, installation becomes impossible at all.

To work around this issue, some installation systems allow you to add disk controller drivers during the initial installation process.

Even if the installation system cannot support all existing hardware, support for that hardware can later be added to the already working server, but before doing so, you must use the disk.

Operating system boot

Placing an operating system running on a hardware platform begins with executing boot routines normally resident in some type of ROM (for example a PC BIOS). In the end a sequence of programs called ***bootloader*** are executed that culminate with RAM loading and operating system execution.

On a PC, the BIOS loads and runs the *bootloader* from the boot disk MBR, in case of the Linux operating system the first *bootloader* then runs a second *bootloader* which is usually **LILO** (*LInux LOader*) or in recent distributions **GRUB** (*GNU GRand Unified Bootloader*).

Whichever *bootloader* runs Linux, the boot process does not end there, once the Linux *kernel* is running it loads and runs the **initial program**, usually **/sbin/init**, which will start all processes that should be running.

For the Linux *kernel* can run and load the initial and remaining program requires an initial file system referred *root*.

Linux – Root file system - /

When the Linux kernel is run by the last *bootloader* it needs to have access to an initial file system called *root*, this information must be passed to the *kernel* at the time it is loaded through arguments, for example:

root=/dev/sda1 **init=/sbin/init**

It means that the *kernel* must mount as root (/) the */dev/sda1* partition and in the end run the initial program */sbin/init*.

The Linux *kernel* also supports the use of network file systems as root (**NFSROOT**) used for example on *diskless* machines.

A widely used alternative is a *ramdrive* with a temporary root file system:

root=/dev/ram0 **initrd=initrd.gz** **init=/sbin/init**

All initial startup of the operating system is done with *ramdrive* which is loaded into memory by *bootloader* along with *kernel*. In the final phase of startup the root file system is usually replaced by another and *ramdrive* is unloaded.

Linux – SWAP partitions

Today's operating systems use virtual memory, this allows the use of external storage devices to store central memory blocks (RAM) and thus make more RAM available to the system than is physically available - an operation called *paging*.
Swapping is a somewhat similar but process-oriented operation, it is about temporarily transferring all data from a process to disk.

On Windows systems virtual memory is based on a file in the system folder, the Unix system uses special disk partitions. Given the slowness of disk operation compared to central memory, it should be seen as a buffer mechanism that prevents memory from running out.

On a Linux system, as long as there is free disk space, it is always possible to add new SWAP partitions because the operating system is capable of simultaneously using several of these partitions, possibly on separate disks..

The amount of central memory in a **server system** should be such that virtual memory is only used very sporadically. For this reason on a Linux server there is no interest in having very large SWAP partitions, generally the rule is to **use an amount of disk space equal to the amount of central memory.**

File system types

Current operating systems support various types of file systems, but not all may have the minimum functionality required for the operating system in question, some key requirements in a single server file system are:

- **ACL (*Access Control List*)** - each object must have an associated ACL defining permissions on the object.
- ***Journaling*** – recording write operations before they are applied, so if there is a serious failure it is not necessary to check the integrity of the entire disk, just the zone that was being written.

Current Linux systems use the fourth extended filesystem (EXT4), Windows systems use NTFS. Both use *journaling*, set permissions (ACLs), and support setting quotas.

NTFS ACLs are much more complex and flexible than EXT4 ACLs, but are partially compatible through the *Portable Operating System Interface* (POSIX) interface provided in NTFS.

Linux – File system table - /etc/fstab

When the Linux *kernel* is loaded into memory one of its first tasks is to mount the root partition (according to the parameters it received). The remaining partitions are mounted later according to the instructions in the **/etc/fstab** configuration file.

In this text file each line represents a partition to mount and has the following format:

device	mounting-point	type	options	dump	fscheckseq

The 6 parameters are separated by spaces or TAB, therefore none of the parameters can contain spaces or TAB:

- ***device*** – device or partition – for exemple **/dev/sdc3**. There are several alternative ways to specify the ***device***, including through a label (*LABEL*) or through a *UUID* (*Universally Unique IDentifier*). The command **blkid** lets you see the UUID and labels that are associated with each existing partition.
- ***mounting-point*** – empty folder where this partition should be mounted – for exemple **/usr/local**

Linux - File system table - /etc/fstab

device	mounting-point	type	options	dump	fscheckseq

- ***type*** – file system type – for example ***ext4***
- ***options*** – mounting options – for example ***ro*** to mount in read-only mode
- ***dump*** – 0 or 1. Zero means the partition should not be included in copies made with the command *dump*. One means it must be included. Only relevant if the administrator intends to use this command for backup purposes (*backup*) and/or arechive
- ***fscheckseq*** – partition repair order (0, 1, 2 , …). In the event of a catastrophic failure, the file systems must be checked at next startup. The value zero means "do not check", normally for *root* partition the value 1 is recommended, being the first one to be checked. For other partitions to be verified, a value of 2 is recommended, which is verified (sequentially or in parallel) after the *root* verification is completed.

Linux - File system table - examples

```
LABEL=Debian    /    ext4    defaults         1      1

/dev/scd0  /media/cdrom0  udf,iso9660  user,noauto,exec,utf8  0  0

/dev/sdb1 /media/disk2 ext2 defaults 0 2

UUID=413eee0c-61ff-4cb7-a299-89d12b075093  /home  ext3  nodev,nosuid,relatime  0  2

UUID=cee15eca-5b2e-48ad-9735-eae5ac14bc90  none   swap   sw   0   0
```

```
UUID=be35a709-c787-4198-a903-d5fdc80ab2f8  /  ext4  relatime,errors=remount-ro  0  1

/dev/sda5          none         swap     defaults        0      0

//server/share  /media/windows  cifs  user=user,uid=1000,gid=100  0     0

server:/share  /media/nfs  nfs  rsize=8192,wsize=8192,noexec,nosuid  0   0
```

Network interfaces configuration

On a server system network interfaces must be configured manually

Using automatic configuration, typically via DHCP creates an external dependency on the system (external DHCP service). On a server for which you want high reliability/availability, all external dependencies are to avoid.

Manual configuration of the network interface requires the collaboration of the network administrator who must provide the various elements:

- IP address (static) + network mask + *default gateway* address
- DNS name servers addresses and local domain name

The only disadvantage of manual configuration is the need for permanent tuning between the network administrator and the server system administrator (often the same). Network configuration changes must be made manually on the server.